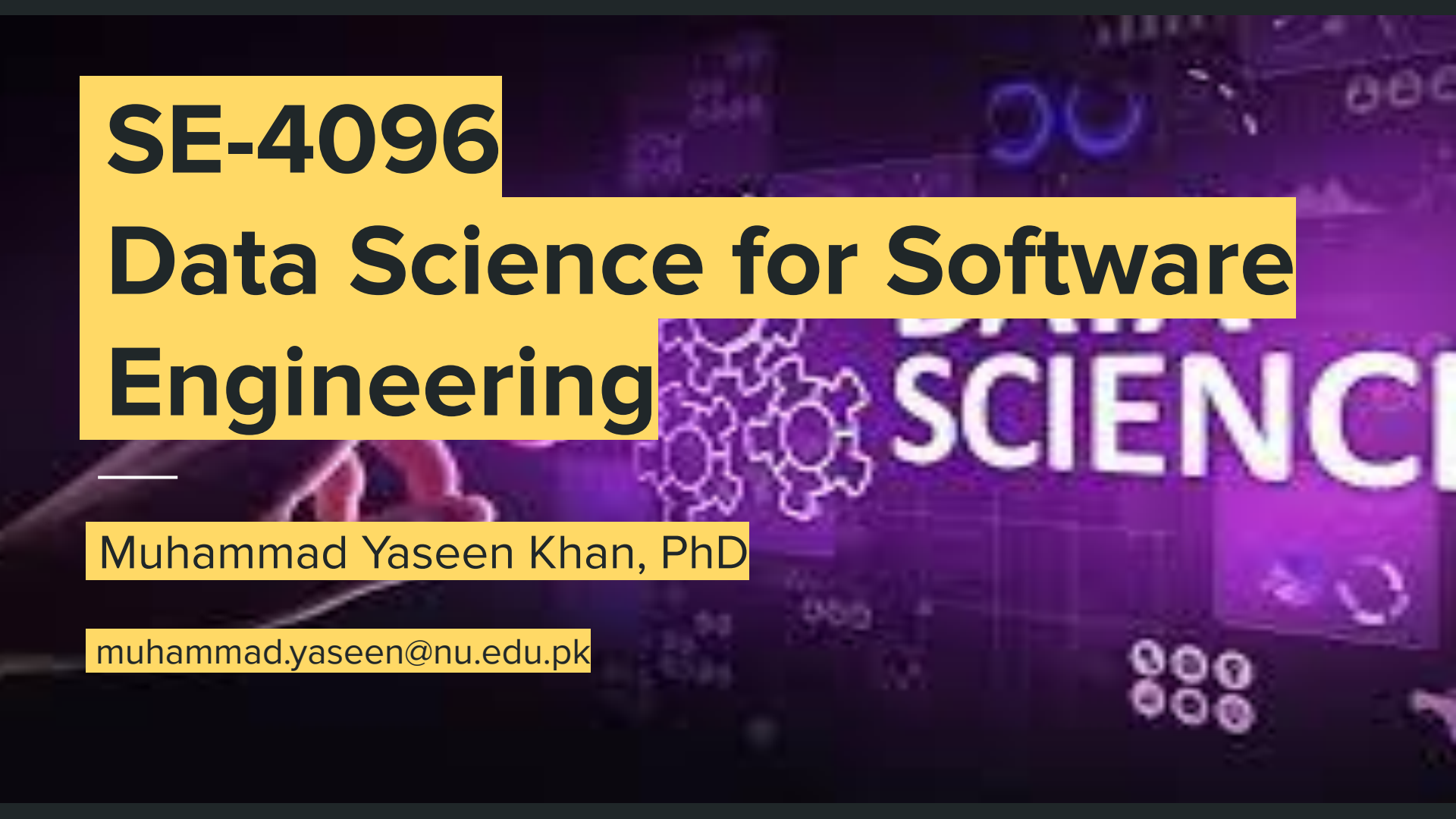




SE-4096

Data Science for Software Engineering

Muhammad Yaseen Khan, PhD

muhammad.yaseen@nu.edu.pk

Dealing with Classification Problem

Today We'll Learn how **k-NN Classifier**
predicts such labels.

Introduction to k-NN

- Key Idea: Similar data points exist close to each other in feature space. Use voting scheme on the nearby/neighbours to predict the outcome
- k-Nearest Neighbors is a supervised learning algorithm.
 - It can be used for classification and regression both.
- For doing so, it transforms the data in vector-space, and compute distance/similarity between the query vector and rest of the vectors in data.
- The data must be in a certain format (vector) where mathematical computation can be performed.
- k-NN is a lazy learning technique.

How to find nearest neighbours

- Distance is a measure of the how far two things are
 - Neighbours are those where the distance is short
- Similarity is a measure of how close two things are
 - Neighbours are those where the similarity is more

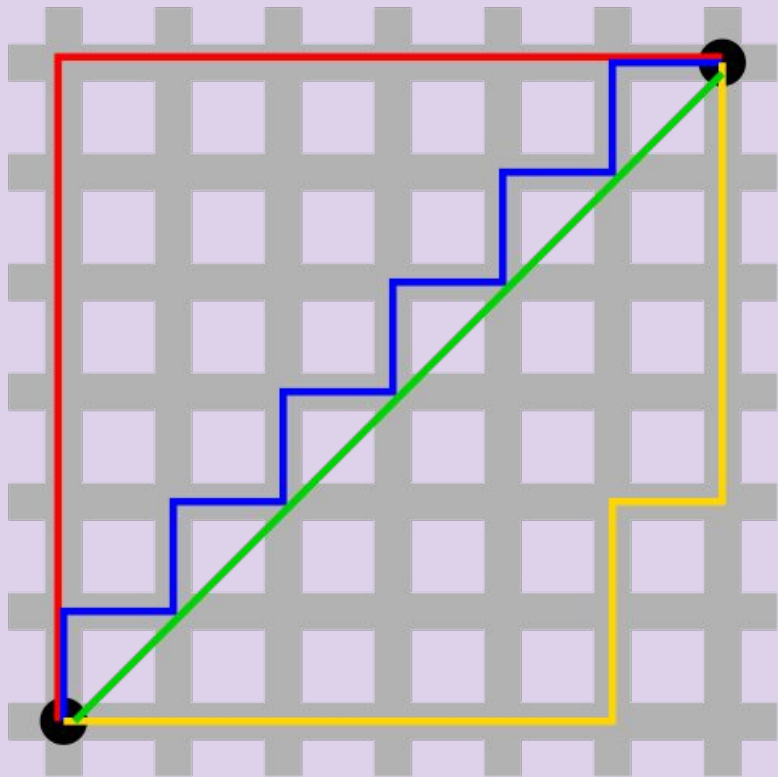
$$\text{distance} \propto \frac{1}{\textit{similarity}}$$

How to find nearest neighbours

There are multiple ways to calculate the distances.

The green line is the shortest distance, namely Euclidean distance.

The rest of the lines are also covering distances, namely Manhattan distance.



How to find nearest neighbours

Euclidean Distance — as the crow flies.

$$\text{euclid}(a, b) = \sqrt{(b_x - a_x)^2 + (b_y - a_y)^2}$$

$$\text{euclid}(a, b) = \sqrt{(b_x - a_x)^2 + (b_y - a_y)^2 + \dots + (b_n - a_n)^2}$$

$$\text{euclid}(a, b) = \sqrt{\sum_{i=0}^n (b_i - a_i)^2}$$

How to find nearest neighbours

Manhattan Distance — counts the intermediate steps.

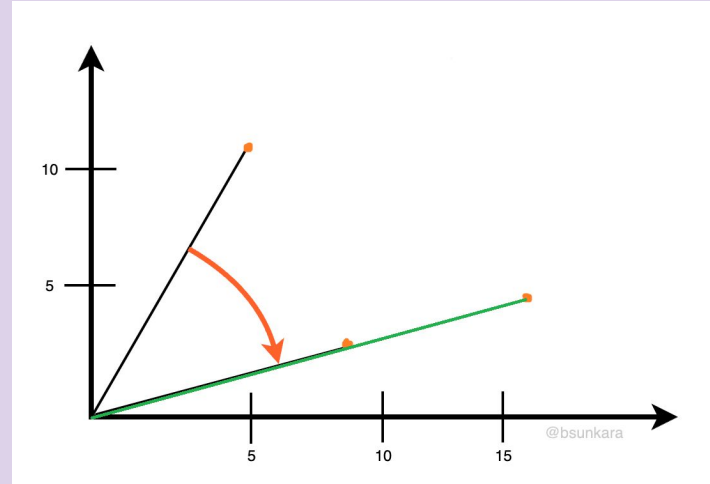
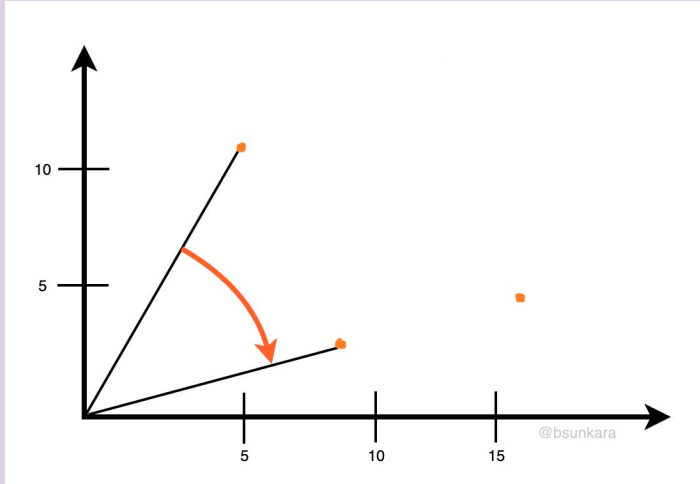
$$\text{manhattan}(a, b) = |b_x - a_x| + |b_y - a_y|$$

$$\text{manhattan}(a, b) = |b_x - a_x| + |b_y - a_y| + \dots + |b_n - a_n|$$

$$\text{manhattan}(a, b) = \sum_{i=0}^n |b_i - a_i|$$

How to find nearest neighbours

Cosine Similarity — cosine of angle between two vectors.



How to find nearest neighbours

Cosine Similarity — cosine of angle between two vectors.

$$\text{cosine similarity}(A, B) \equiv \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}},$$

One-Hot Encoding

- Categorical attributes must be converted to numbers.
- Example:
 - Language = {Java, Python, C++}
 - Java $\rightarrow (1,0,0)$
 - Python $\rightarrow (0,1,0)$
 - C++ $\rightarrow (0,0,1)$

Defect Prediction Dataset

Module	Language	LOC	TestCoverage	BugCount	DefectRisk
M1	Java	1200	80	2	Low
M2	Python	900	60	7	High
M3	Java	1500	75	3	Low
M4	C++	2000	50	9	High
M5	Python	1100	85	1	Low

Here:

- **Language** is a **categorical feature**
- ML algorithms like **k-NN** **require numeric features**
- The **Language** column contains 3 categories:
- Java, Python, C++

Transformed Defect Prediction Dataset

We create **3 new binary columns**:

- Language_Java
- Language_Python
- Language_CPP

Module	Lang_Java	Lang_Python	Lang_CPP	LOC	Coverage	Bugs	DefectRisk
M1	1	0	0	1200	80	2	Low
M2	0	1	0	900	60	7	High
M3	1	0	0	1500	75	3	Low
M4	0	0	1	2000	50	9	High
M5	0	1	0	1100	85	1	Low

k-NN Algorithm step by step

Simplified Steps of the Algorithm

1. **Choose a value for k**

Decide how many nearby data points (neighbors) you want the algorithm to consider.

2. **Calculate the distance**

Measure the distance between the new data point (query) and all the training data points.

3. **Sort the distances**

Arrange all training points from **closest to farthest** based on the distance.

4. **Select the nearest neighbors**

Pick the **k closest data points**.

5. **Check their classes**

Look at the class labels of these k neighbors.

6. **Predict the class**

Assign the class that appears **most frequently** among the neighbors to the new data point.

k-NN Algorithm step by step

Simplified Steps of the Algorithm

1. **Choose a value for k**

Decide how many nearby data points (neighbors) you want the algorithm to consider.

2. **Calculate the distance**

Measure the distance between the new data point (query) and all the training data points.

3. **Sort the distances**

Arrange all training points from **closest to farthest** based on the distance.

4. **Select the nearest neighbors**

Pick the **k closest data points**.

5. **Check their classes**

Look at the class labels of these k neighbors.

6. **Predict the value**

Assign the value that is the **average of k nearest neighbours** to the new data point.

Hands-on Classification with k-NN

Module	Lang_Java	Lang_Python	Lang_CPP	LOC	Coverage	Bugs	DefectRisk
M0	1	1	0	1100	60	4	?

Value of k = 3

Module	Lang_Java	Lang_Python	Lang_CPP	LOC	Coverage	Bugs	DefectRisk
M0	1	1	0	1100	?	4	Low

Lab:

- Introduction to
 - Train Test Splits
 - LOO, x-validation
 - Naive Bayes with scikit-learn
 - Evaluation Metrics